

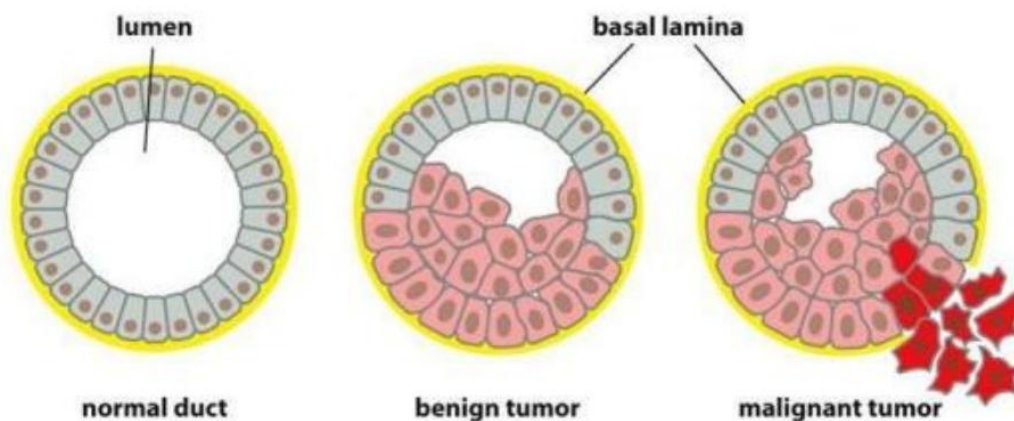
1. Introduction

Breast cancer is the most common cancer among women and the second most dangerous cancer after lung cancer, according to the World Health Organization. According to the research, a total of 627,000 women died from breast cancer in 2018, accounting for 15% of all cancer deaths among women. People should see an oncologist if they experience any symptoms. Breast ultrasound, diagnostic mammogram, magnetic resonance imaging (MRI), and biopsies can all help doctors identify breast cancer. Doctors may recommend additional tests or treatment based on the results of these tests. Early detection of breast cancer is crucial. If cancer is predicted at an early stage, the patient's chances of recovery may keep improving. Another method for predicting breast cancer is to use machine learning algorithms to predict abnormal tumours. As a result, research has been undertaken in order to properly diagnose and classify patients into malignant and benign groups.

The three types of tumours are as follows:

- a. Benign tumours are not cancerous, cannot spread, and develop slowly. Even if they are removed by surgeons, they cannot damage the human body.
- b. Premalignant tumours are not cancerous, but they have the ability to become so.
- c. Malignant tumours are cancerous and can grow quickly across the body.

Benign vs Malignant Tumor



2. Objective

Cancer classification can be performed using benign or malignant cells in machine learning, and while this may greatly boost our ability to predict prognosis in cancer patients, no progress has been made in their use in clinics. However, before gene expression profiling can be used in medical care, wider data sets and more thorough evaluation are needed. As a result, our goal is to create a predictive framework that can determine the occurrence of breast cancer based on large data sets.

3. Data Description

Data Information:

- ID number
- Diagnosis (M = malignant, B = benign)

Ten real-valued features are computed for each cell nucleus:

- radius (mean of distances from center to points on the perimeter)
- texture (standard deviation of gray-scale values)
- perimeter
- area
- smoothness (local variation in radius lengths)
- compactness ($\text{perimeter}^2 / \text{area} - 1.0$)
- concavity (severity of concave portions of the contour)
- concave points (number of concave portions of the contour)
- symmetry
- fractal dimension ("coastline approximation" - 1)

4. Exploratory Data Analysis

Importing Libraries and Dataset

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import RandomForestClassifier

from sklearn.naive_bayes import GaussianNB

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, classification_report

from sklearn.svm import SVC

from sklearn import metrics

df = pd.read_csv("data.csv")
```

```
df.shape
```

```
(569, 33)
```

```
df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430

5 rows x 33 columns

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   id                                     569 non-null    int64
1   diagnosis                             569 non-null    object
2   radius_mean                           569 non-null    float64
3   texture_mean                           569 non-null    float64
4   perimeter_mean                         569 non-null    float64
5   area_mean                             569 non-null    float64
6   smoothness_mean                       569 non-null    float64
7   compactness_mean                      569 non-null    float64
8   concavity_mean                        569 non-null    float64
9   concave points_mean                   569 non-null    float64
10  symmetry_mean                         569 non-null    float64
11  fractal_dimension_mean                569 non-null    float64
12  radius_se                             569 non-null    float64
13  texture_se                             569 non-null    float64
14  perimeter_se                           569 non-null    float64
15  area_se                               569 non-null    float64
16  smoothness_se                         569 non-null    float64
17  compactness_se                        569 non-null    float64
18  concavity_se                          569 non-null    float64
19  concave points_se                     569 non-null    float64
20  symmetry_se                           569 non-null    float64
21  fractal_dimension_se                  569 non-null    float64
22  radius_worst                          569 non-null    float64
23  texture_worst                         569 non-null    float64
24  perimeter_worst                       569 non-null    float64
25  area_worst                            569 non-null    float64
26  smoothness_worst                      569 non-null    float64
27  compactness_worst                     569 non-null    float64
28  concavity_worst                       569 non-null    float64
29  concave points_worst                  569 non-null    float64
30  symmetry_worst                        569 non-null    float64
31  fractal_dimension_worst                569 non-null    float64
32  Unnamed: 32                           0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

Data has one unnamed column which has null values. Need to drop that column.

```
df = df.dropna(axis='columns')
```

Checking for count and unique values of categorical features

```
df.describe(include="O")
```

diagnosis	
count	569
unique	2
top	B
freq	357

```
df.diagnosis.unique()
array(['M', 'B'], dtype=object)
```

The above result shows that there is only single categorical feature and its unique values are B and M.

Count of unique values

```
df.diagnosis.value_counts()
```

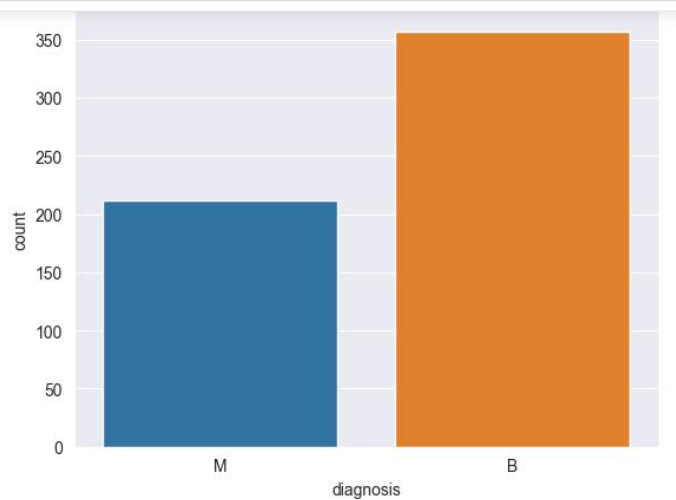
```
diagnosis
B      357
M      212
Name: count, dtype: int64
```

Data Visualization

```
import matplotlib.pyplot as plt
import seaborn as sns

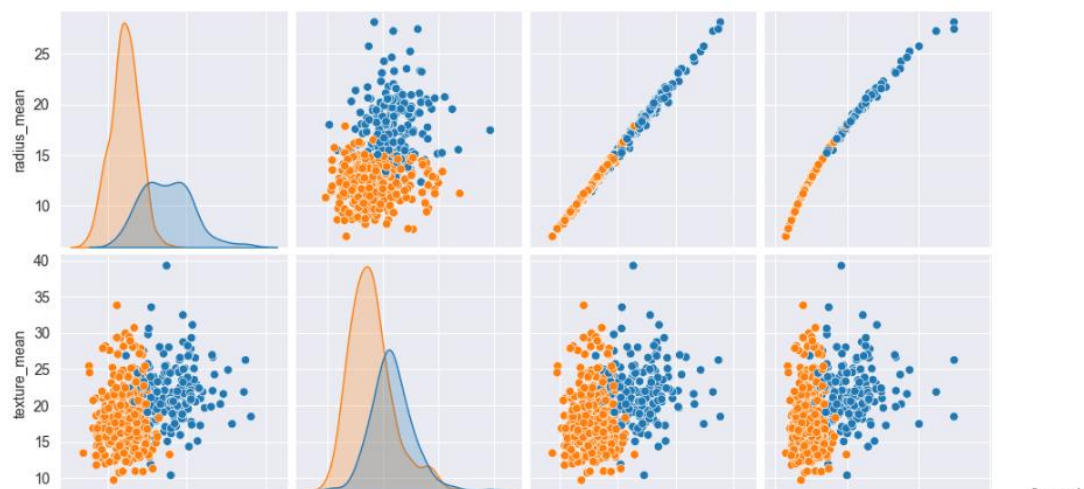
%matplotlib inline
sns.set_style('darkgrid')
```

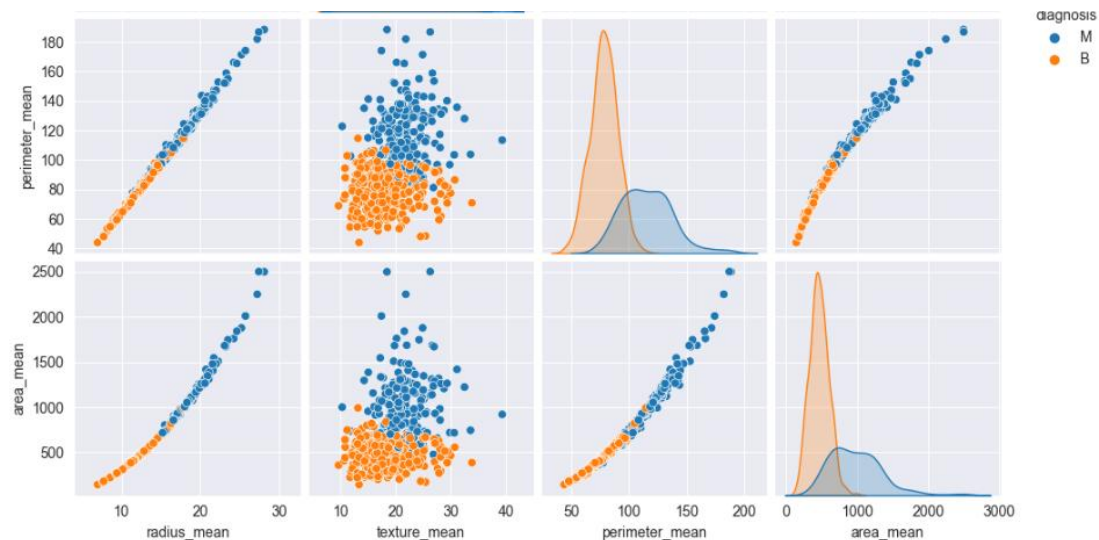
```
sns.countplot(df, x="diagnosis")
```



```
plt.figure(figsize=(15, 5))
cols = ["diagnosis", "radius_mean", "texture_mean", "perimeter_mean", "area_mean"]

sns.pairplot(df[cols], hue="diagnosis")
plt.show()
```





5. Data Preprocessing

Feature Encoding

There is one categorical feature, so we need to convert it into numeric values using LabelEncoder from sklearn.preprocessing packages. LabelEncoder can be used to normalize features.

```
from sklearn.preprocessing import LabelEncoder
```

```
labelencoder_Y = LabelEncoder()
df.diagnosis = labelencoder_Y.fit_transform(df.diagnosis)
```

```
df.head(2)
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_m
0	842302	1	17.99	10.38	122.8	1001.0	0.11840	0.27760	0.1
1	842517	1	20.57	17.77	132.9	1326.0	0.08474	0.07864	0.1

2 rows x 32 columns

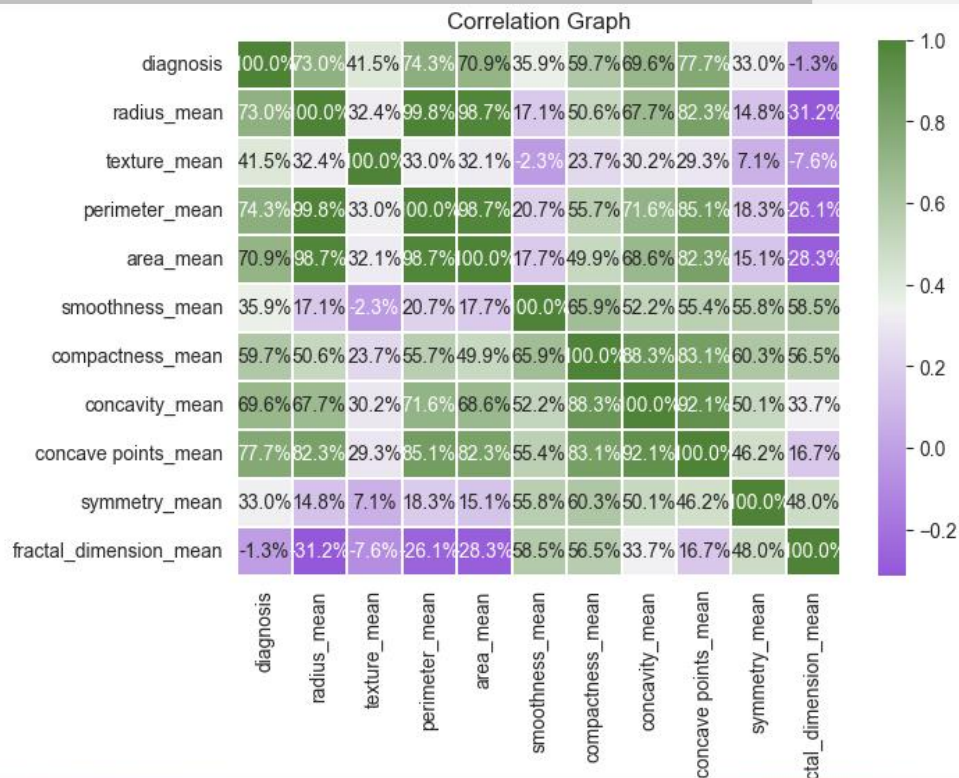
It is see in this output , that categorical values are converted into 0 and 1.

Correlation between target variable and other features

```
cols = ['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
        'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
        'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean']
print(len(cols))
df[cols].corr()
```

11

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean
diagnosis	1.000000	0.730029	0.415185	0.742636	0.708984	0.358560	0.596534	0.696360
radius_mean	0.730029	1.000000	0.323782	0.997855	0.987357	0.170581	0.506124	0.676764
texture_mean	0.415185	0.323782	1.000000	0.329533	0.321086	-0.023389	0.236702	0.302418
perimeter_mean	0.742636	0.997855	0.329533	1.000000	0.986507	0.207278	0.556936	0.716136
area_mean	0.708984	0.987357	0.321086	0.986507	1.000000	0.177028	0.498502	0.685983
smoothness_mean	0.358560	0.170581	-0.023389	0.207278	0.177028	1.000000	0.659123	0.521984
compactness_mean	0.596534	0.506124	0.236702	0.556936	0.498502	0.659123	1.000000	0.883121
concavity_mean	0.696360	0.676764	0.302418	0.716136	0.685983	0.521984	0.883121	1.000000
concave points_mean	0.776614	0.822529	0.293464	0.850977	0.823269	0.553695	0.831135	0.921391
symmetry_mean	0.330499	0.147741	0.071401	0.183027	0.151293	0.557775	0.602641	0.500667
fractal_dimension_mean	-0.012838	-0.311631	-0.076437	-0.261477	-0.283110	0.584792	0.565369	0.336783



Splitting the dataset

The training phase extracts features from the dataset, and the testing phase determines how the effective model performs for prediction. The dataset is split into two sections: training and testing. We have trained 80% of the data and tested 20% of the data.

```
prediction_feature = [ "radius_mean", 'perimeter_mean', 'area_mean', 'symmetry_mean', 'compactness_mean',
targeted_feature = 'diagnosis'

X = df[prediction_feature]
y = df.diagnosis

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=10)
```

Feature Scaling

Standardization is a scaling technique wherein it makes the data scale-free by converting the statistical distribution of the data into the below format:

mean - 0 (zero)
standard deviation - 1

Python sklearn library offers us with StandardScaler() function to standardize the data values into a standard format.

```
sc = StandardScaler()

X_train = sc.fit_transform(X_train)
X_test = sc.fit_transform(X_test)
```

6. Data Model Building

The dataset is evaluated using various classification algorithms listed below:

- Logistic Regression
- Random Forest Classifier
- Decision Tree Classifier
- Support Vector Classifier
- Naive Bayes
- K Nearest Neighbors

A common function is build to capture the metrics of algorithms.

```
def model_building(model, X_train, X_test, y_train, y_test):
    """
    Model Fitting, Prediction And Other stuff
    return ('score', 'accuracy_score', 'predictions' )
    """

    model.fit(X_train, y_train)
    score = model.score(X_train, y_train)
    predictions = model.predict(X_test)
    accuracy = accuracy_score(predictions, y_test)

    return (score, accuracy, predictions)
```

```
models_list = {
    "LogisticRegression" : LogisticRegression(),
    "RandomForestClassifier" : RandomForestClassifier(n_estimators=10, criterion='entropy', random_state=5),
    "DecisionTreeClassifier" : DecisionTreeClassifier(criterion='entropy', random_state=0),
    "SVC" : SVC(),
    "NaiveBayes" : GaussianNB(),
    "KNN" : KNeighborsClassifier(n_neighbors= 5)
}
```

```

df_prediction = []
confusion_matrixs = []
df_prediction_cols = [ 'model_name', 'score', 'accuracy_score' , "accuracy_percentage"]

for name, model in zip(list(models_list.keys()), list(models_list.values())):

    (score, accuracy, predictions) = model_building(model, X_train, X_test, y_train, y_test )

    print("\n\nClassification Report of '"+ str(name), "'\n")

    print(classification_report(y_test, predictions))

    df_prediction.append([name,score, accuracy, "{0:.2%}".format(accuracy)])

    # For Showing Metrics
    confusion_matrixs.append(confusion_matrix(y_test, predictions))

df_pred = pd.DataFrame(df_prediction, columns=df_prediction_cols)

```

7. Test Results

Classification Report of 'LogisticRegression '

	precision	recall	f1-score	support
0	0.97	0.94	0.96	124
1	0.90	0.94	0.92	64
accuracy			0.94	188
macro avg	0.93	0.94	0.94	188
weighted avg	0.94	0.94	0.94	188

Classification Report of 'RandomForestClassifier '

	precision	recall	f1-score	support
0	0.93	0.92	0.93	124
1	0.85	0.88	0.86	64
accuracy			0.90	188
macro avg	0.89	0.90	0.89	188
weighted avg	0.91	0.90	0.90	188

Classification Report of 'DecisionTreeClassifier '

	precision	recall	f1-score	support
0	0.95	0.89	0.92	124
1	0.81	0.91	0.85	64
accuracy			0.89	188
macro avg	0.88	0.90	0.88	188
weighted avg	0.90	0.89	0.89	188

Classification Report of 'SVC '

	precision	recall	f1-score	support
0	0.98	0.95	0.96	124
1	0.91	0.95	0.93	64
accuracy			0.95	188
macro avg	0.94	0.95	0.95	188
weighted avg	0.95	0.95	0.95	188

Classification Report of 'NaiveBayes '

	precision	recall	f1-score	support
0	0.98	0.93	0.95	124
1	0.87	0.97	0.92	64
accuracy			0.94	188
macro avg	0.93	0.95	0.94	188
weighted avg	0.95	0.94	0.94	188

Classification Report of 'KNN '

	precision	recall	f1-score	support
0	0.97	0.92	0.95	124
1	0.86	0.95	0.90	64
accuracy			0.93	188
macro avg	0.92	0.94	0.92	188
weighted avg	0.94	0.93	0.93	188

df_pred				
	model_name	score	accuracy_score	accuracy_percentage
0	LogisticRegression	0.900262	0.941489	94.15%
1	RandomForestClassifier	0.992126	0.904255	90.43%
2	DecisionTreeClassifier	1.000000	0.893617	89.36%
3	SVC	0.913386	0.952128	95.21%
4	NaiveBayes	0.902887	0.941489	94.15%
5	KNN	0.934383	0.930851	93.09%

8. Conclusion

A machine learning model to predict breast cancer was developed using various classification algorithms like Logistic Regression, Random Forest Classifier, Decision Tree Classifier, Support Vector Classifier, Naive Bayes, K Nearest Neighbors. Of all the algorithms **Support Vector classifier** achieved an accuracy of 95.21% on this dataset, indicating that it is a highly effective model.